

Secret Status: Top Secret () Secret () Restrict () Public (☒)

RKNanoD Wireless Audio SDK

User Manual

(Version 1.2)

Product R&D Dept.III

Rockchip Electronics Co.,Ltd

www.rock-chips.com

Status: [] development [☒] public [] modify	File Tag:	RKNanoD Wireless Audio SDK User Manual	
	Version:	1.2	
	Author:	Wang Ruoming	
	Data:	2016 年 3 月 8 日	
	Verify:	Zheng Yongzhi	2016 年 4 月 15 日

Version History

Version	Author	Date	Remarks
1.0	Wang Ruoming	2016-03-08	Original
1.1	Wang Ruoming	2016-4-15	Update
1.2	Wang Ping	2016-5-4	Update

目录

1	SDK 概要.....	5
1.1	SDK 概述.....	5
1.2	设计目标.....	5
2	SDK 文件目录.....	5
2.1	总目录组织结构.....	5
2.2	子目录具体结构.....	6
2.2.1	工程目录.....	6
2.2.2	应用程序目录.....	6
2.2.3	Driver 目录	7
2.2.4	OS 目录.....	9
2.2.5	System 目录	9
2.2.6	Wireless 目录.....	9
3	SDK 系统.....	11
3.1	系统框图.....	11
3.2	应用层.....	11
3.3	本地音乐播放软件架构.....	12
3.4	DLNA 播放器软件架构.....	13
3.5	蓝牙播放器软件架构.....	14
3.6	线程管理器.....	14
3.7	设备驱动管理.....	15
3.8	OS 接口.....	15
3.9	音频解码的 AB 核通信	15
4	SDK 配置、编译与固件下载.....	16
4.1	SDK 配置.....	16
4.2	SDK 编译.....	17
4.2	固件生成.....	18
4.3	固件下载.....	19
5	SDK 操作指南.....	20

5.1	开机流程.....	20
5.2	操作说明.....	20
5.2.1	连接 WIFI.....	20
5.2.2	DLNA 播放器操作.....	21
5.2.3	第三方播放器操作.....	22
5.2.4	蓝牙播放器操作.....	23
5.2.5	本地播放器操作.....	23
5.3	无屏音箱配置与操作.....	24
5.3.1	无屏音箱代码配置.....	24
5.3.3	蓝牙无屏音箱操作流程.....	27
6	SDK 辅助工具.....	29
6.1	shell 命令集合.....	29
6.1.1	音乐 shell 命令.....	29
6.1.2	USB shell 命令.....	29
6.1.3	WIFI、DLNA、shell 命令.....	30
6.1.4	蓝牙 shell 命令.....	30
6.1.4	系统 shell 命令.....	30
6.1.5	线程 shell 命令.....	31
6.2	shell 命令调试实例.....	31
6.2.1	音乐播放 shell 操作.....	31

1 SDK 概要

1.1 SDK 概述

RKNanoD Wireless Audio SDK 是一个基于 FreeRTOS 内核的实时操作系统，采用内核隔离技术，可以在特定需求下更换系统内核。拥有自主设计的设备驱动框架、线程框架、GUI 框架、段重载机制、系统休眠机制，以及类 shell 调试平台。

目前 SDK 已经支持 WIFI、蓝牙等相关功能，包括 MP3 播放器，DLNA 播放器，第三方应用播放器，蓝牙播放器等功能。

1.2 设计目标

RKNanoD Wireless Audio SDK 是为了解决广大无线音频播放的应用需求，提供一个开发周期短、可靠性高、品质优良的产品解决方案。

2 SDK 文件目录

2.1 总目录组织结构

RKNanoD Wireless Audio SDK 平台是一个包含了嵌入式实时操作系统（FreeRTOS），用户图形界面接口（GUI），文件系统（FS），音频编解码器（CODEC），板级支持包（BSP），WiFi 协议栈、蓝牙协议栈以及一系列应用程序例如 MP3 播放器、DLAN 播放器、第三方应用播放器等的软件平台，有利于不同应用程序的移植与编写，其文件结构如下：

RKNanoD_Wireless_Audio_SDK		文件说明
RKNanoD_Wireless_Audio_SDK	App	应用程序代码及应用相关的工程配置等文件
	BBSystem	B 核相关代码
	Codecs	编解码相关代码
	Cpu	CPU 相关代码

	Driver	驱动相关代码
	Gui	GUI 相关代码
	OS	SDK OS 相关代码
	System	系统服务相关代码
	Wireless	无线播放器相关代码
	rkos_doc	SDK 文档目录
	Tools	SDK 开发辅助工具

2.2 子目录具体结构

2.2.1 工程目录

应用程序目录		文件说明
APP	All	包含所有功能的工程目录
	Rki6000_Demo	Rki6000 工程目录

2.2.2 应用程序目录

文件说明			
All/Rki6000_Demo	Project		编译工程目录
	Development_tools	firmware_generate_eMMC	EMMC 固件生成工具
		firmware_generate_SPI	SPI 固件生成工具
		firmware_upgrade	固件烧写工具
	Audio		音频控制框架相关代码
	Browser		资源管理相关代码

	FileStream	文件流控相关代码
	main_task	主界面相关代码
	Media	多媒体相关代码
	MediaLibrary	媒体库相关代码
	music	音乐播放 UI 相关代码
	Record	录音相关代码
	SystemSet	系统设置相关代码
	USB	充电检测相关代码
	Resource	图片、菜单等资源数据
	Scatter	脚本控制目录
	AppInclude.h	APP 文件的头文件
	board_main_v10_20150126_config.h	对应硬件开发板配置
	board_main_v21_20150515_config.h	对应硬件开发板配置
	board_main_v22_20150727_config.h	对应硬件开发板配置
	BSP.c	板级配置接口代码
	BSP.h	
	BspConfig.h	SDK 功能配置头文件
	FreeRTOSConfig.h	RTOS 配置头文件
	Main.c	主程序入口
	Main.h	

2.2.3 Driver 目录

设备 Driver 目录		文件说明
Driver	AD	ADC 设备驱动
	Audio	Audio 设备驱动
	Bcore	B 核设备驱动

	Display	Display 设备驱动
	DMA	DMA 设备驱动
	EMMC	EMMC 设备驱动
	FIFO	FIFO 设备驱动
	File	File 设备驱动
	I2C	I2C 设备驱动
	I2S	I2S 设备驱动
	Key	Key 设备驱动
	LUN	LUN 设备驱动
	MailBox	MailBox 设备驱动
	MSG	Msg 设备驱动
	Partion	Partion 设备驱动
	PWM	PWM 设备驱动
	ROCKCODEC	RK ACODEC 设备驱动
	SD	SD 卡设备驱动
	Sdio	Sdio 设备驱动
	SdMmc	SdMmc 设备驱动
	SPI	SPI 设备驱动
	SPIFlash	SPIFlash 设备驱动
	SpiNor	SpiNor 设备驱动
	Timer	Timer 设备驱动
	Uart	Uart 设备驱动
	USB	USB OTG 设备驱动
	USBMsc	USB MSC 设备驱动
	Volume	Volume 设备驱动
	Vop	Vop 设备驱动
	WatchDog	WatchDog 设备驱动
	DeviceInclude.h	设备头文件

2.2.4 OS 目录

OS 目录		文件说明
OS	DeviceManager	设备管理器代码
	FwAnalysis	Modele Overlay 代码
	Include	操作系统头文件代码
	Plugin	线程管理，设备管理，RKOS 代码
	power	系统休眠与唤醒代码
	Source	与 FreeRTOS 有关的任务，堆栈，队列，线程，定时器代码

2.2.5 System 目录

System 目录		文件说明
System	Delay	系统延迟相关代码
	ModuleOverlay	读写系统参数相关代码
	Shell	Shell 调用相关代码
	syssever	电源管理相关代码
	UsbServer	Usb 服务相关代码

2.2.6 Wireless 目录

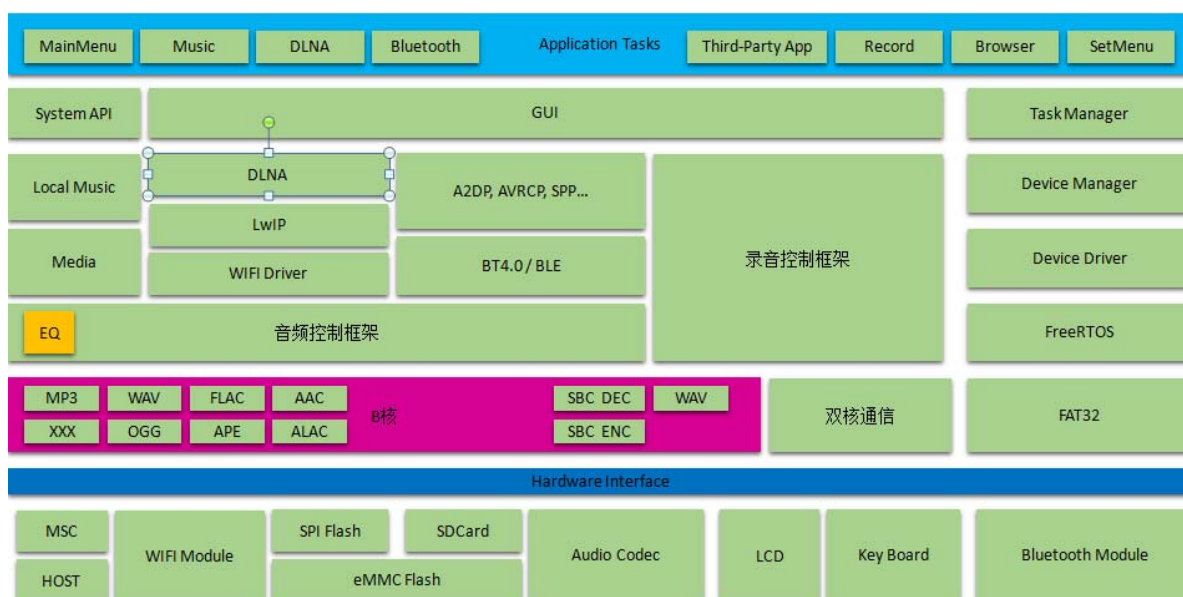
Web 目录		文件说明
	bluetooth	蓝牙代码
	channels	5.1 声道音箱上层协议代码

	dlna	Dlna 代码
	Lwip	Lwip 代码
	Task	Web 任务代码
	WebServer	Lwip 网页服务 httpd 代码
	RKI6000	RKI6000 驱动相关代码
	WICE	AP 系列 wifi 芯片驱动相关代码

3 SDK 系统

3.1 系统框图

如下图所示：RKNanoD Wireless Audio SDK 是一个基于 FreeRTOS 内核的实时操作系统，主要功能包括应用层 APP、WiFi/BT 协议层、OS 调度层、设备驱动层以及 B 核音频解码部分等。SDK 软件支持本地音乐播放、DLNA 播放、第三方应用播放器以及蓝牙播放、录音等功能。如下图所示



如图所示：

1. Third-Part App 第三方应用程序的接口，用户可以根据自己的需要增加应用程序；
2. XXX 是编码器接口，用户可以根据自己的需求增加音频编码程序；

3.2 应用层

目前 RKNanoD Wireless Audio SDK 在应用上拥有主菜单、MP3 播放器、DLNA 播放器、蓝牙播放器、第三方播放器接口、资源管理器、设置菜单以及 USB 存储。

3.3 本地音乐播放软件架构

本地音乐播放软件架构如下图所示：

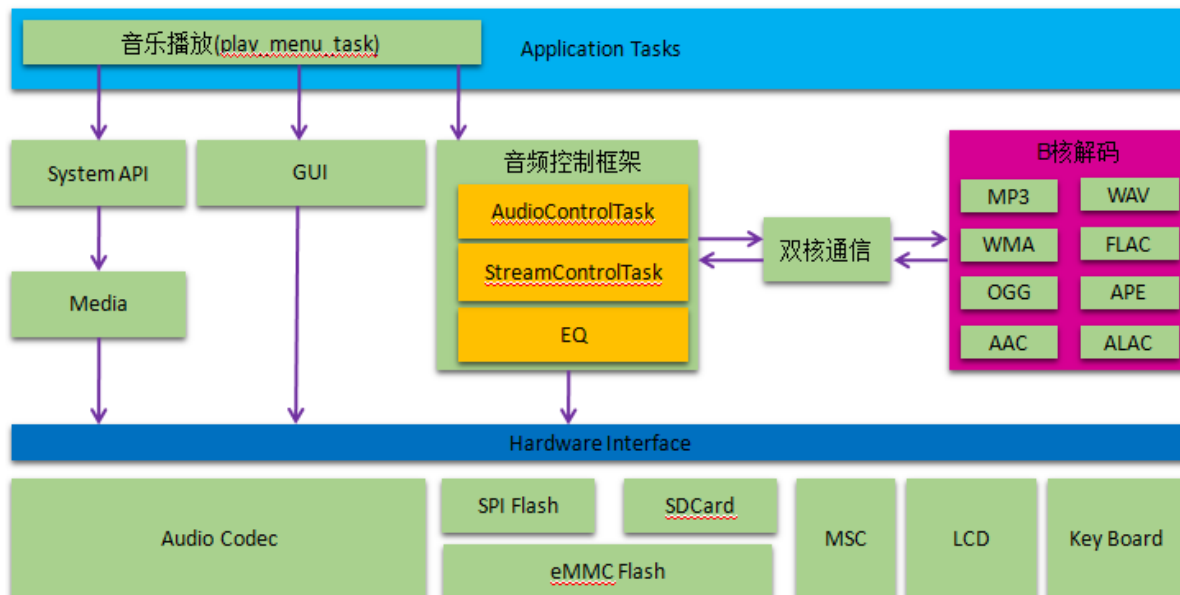


图 3.4

由上图可知本地音乐播放功能主要支持 MP3、WAV、FLAC、OGG、APE、AAC、ALAC 等音频文件格式。

本地音乐播放软件系统框架由应用层“play_menu_task”任务，音频控制层“AudioControlTask, StreamControlTask”，以及系统任务管理、设备管理、底层驱动和 B 核解码等多部分组成。其中：

play_menu_task: 负责音乐播放界面显示和操作控制；

AudioControlTask: 负责音频操作控制接口；

StreamControlTask: 负责 B 核解码数据流交互，获取文件数据传递给 B 核解码，并从 B 核获取解码后的数据；

3.4 DLNA 播放器软件架构

DLNA 音乐播放软件架构如下图所示：

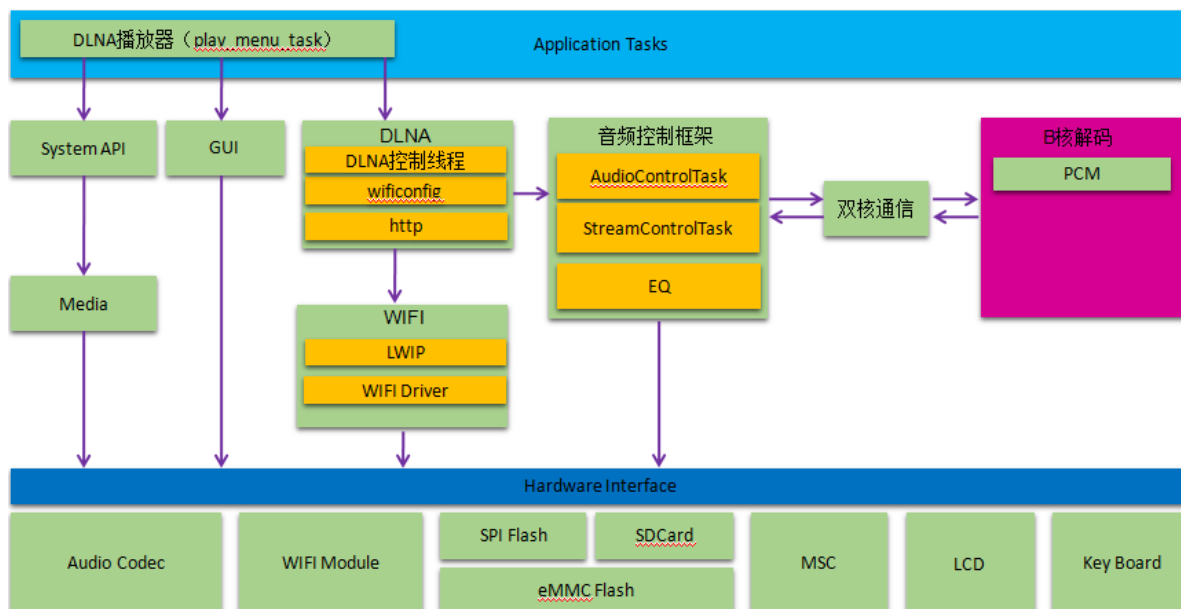


图 3.6

由上图可知 DLNA 音乐播放软件系统框架由应用层“play_menu_task”任务，“Dlna”任务，“wifi”模块，音频控制层“AudioControlTask, StreamControlTask”，以及系统任务管理、设备管理、底层驱动和 B 核解码等多部分组成。其中：

play_menu_task: 负责音乐播放界面显示和操作控制；

Dlna: 负责 Dlna 协议的解析与处理；

wifi: 负责 wifi 驱动以及 LWIP 协议的解析与处理；

AudioControlTask: 负责音频操作控制接口；

StreamControlTask: 负责 B 核解码数据流交互，获取文件数据传递给 B 核解码，并从 B 核获取解码后的数据；

3.5 蓝牙播放器软件架构

蓝牙音乐播放软件架构如下图所示：

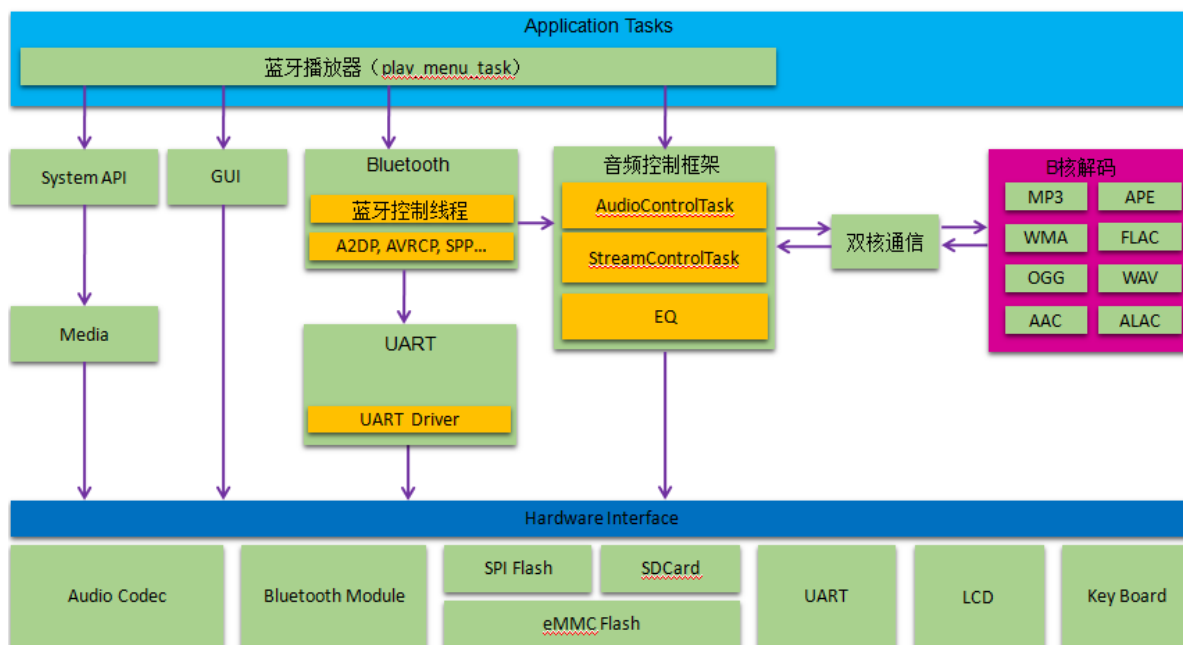


图 3.7

由上图可知蓝牙音乐播放软件系统框架由应用层“play_menu_task”任务，“BluetoothControl”任务，音频控制层“AudioControlTask, StreamControlTask”，以及系统任务管理、设备管理、底层驱动和 B 核解码等多部分组成。其中：

play_menu_task: 负责音乐播放界面显示和操作控制；

BluetoothControl: 负责 Bluetooth 协议的解析与处理，功能包括 A2DP, AVRCP 等功能；

UART: 支持 HCI5 标准蓝牙传输接口，负责蓝牙模块与 CPU 的数据通信；

AudioControlTask: 负责音频操作控制接口；

StreamControlTask: 负责 B 核解码数据流交互，获取文件数据传递给 B 核解码，并从 B 核获取解码后的数据；

3.6 线程管理器

在 RKNanoD Wireless Audio SDK 中，RKNanoD 通过线程管理器进行线程管理，线程管理器掌管着所有线程的创建与删除，用户在使用过程中新增加线程或者删除线程，必须通过线程管理器进行操作和管理。

3.7 设备驱动管理

RKNanoD 采用设备驱动管理框架进行设备驱动的管理，设备管理任务 DeviceManagerTask.c 对所有的设备建立统一的设备链路，再通过设备链路对所需要的设备依次建立，创建设备的具体函数实现都在 Device.c 中。

RknanoD 的设备驱动框架将硬件操作通过 API 的方式提供给设备驱动调用，不允许设备驱动直接访问寄存器，所有有关硬件设备寄存器的操作都通过硬件抽象层进行隔离，方便在不同的硬件平台进行移植。

3.8 OS 接口

RKNanoD Wireless Audio SDK 目前使用了 FreeRTOS 的内核，但是不允许开发人员直接调用 FreeRTOS 的接口。RKNanoD 对 FreeRTOS 的接口通过内核隔离技术进行分割，以满足 RKNanoD 系统可以针对不同的 OS 进行移植。

3.9 音频解码的 AB 核通信

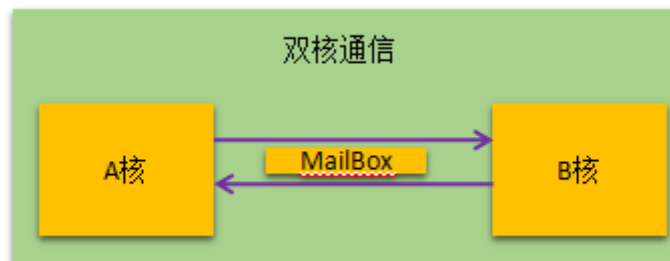


图 3.13

RKNanoD 是一个双 ARM-Cortex M3 内核芯片，另外 RKNanoD 支持硬件加速器功能，可以满足各种格式的解码需求。RknanoD 通过 B 核配合硬件加速器进行音频算法处理，A 核作为系统核进行任务调度、文件管理、协议处理等工作。磁盘、无线等传输的音频数据，通过 A 核进行获取后传递给 B 核解码，B 核解码完成之后将解码后数据传递给 A 核进行后处理和 ACODEC 输出。A 核与 B 核是通过 MailBox 进行通信控制，AB 核的数据交互采用芯片内部系统总线进行访问管理。

4 SDK 配置、编译与固件下载

4.1 SDK 配置

RKNanoD Wireless Audio SDK 是一个支持 MP3 播放器、网络播放器、蓝牙播放器的综合软件开发包，同时兼容 SPI Flash、eMMC Flash 等存储设备。用户针对不同的产品需求，需要对 SDK 进行相应的配置，以满足各自需求。SDK 的配置文件在 Bsp\EVK_V2.0\BspConfig.h 中：

Flash 存储设备配置：如下图所示，如果使用的是 EMMC Flash，打宏 “_EMMC_BOOT_” 开关；如果是使用 SPI Flash，可以打开宏 “_SPI_BOOT_”。

```
//boot switch
//#define _SPI_BOOT_
#define _EMMC_BOOT_
//#define _NAND_BOOT_
//#define _UART_BOOT_
//#define _USB_BOOT_
```

SDK 功能配置：SDK 支持本地 MP3 播放器，蓝牙播放，DLNA 播放器，第三方播放器，可以根据不同的功能需求进行相应的配置，如下图所示：

```
#define _BLUETOOTH_
#define _WIFI_FOR_SYSTEM_
//#define _WIFI_
#define _ADC_KEY_
#define _USE_GUI_

#ifdef _ADC_KEY_
#define _BATTERY_
#endif
```

DLNA、蓝牙设备名称修改方式：由于 SDK 使用了 Rockchip 统一的 DLNA、蓝牙的设备名称和地址，用户在开发产品时需要对该设备名称进行修改，主要修改的地方有：

DLNA：..\Bsp\EVK_V2.0\Bsp.c 的 GetDlnaDeviceName () 函数中，修改以下部分：

```
*****
_BSP_EVK_V20_BSP_COMMON_
COMMON API void GetDlnaDeviceName (uint8 * name)
{
    strcpy(name, "RKNanoD Media Renderer [SPEAKER 1314520]");
}
```



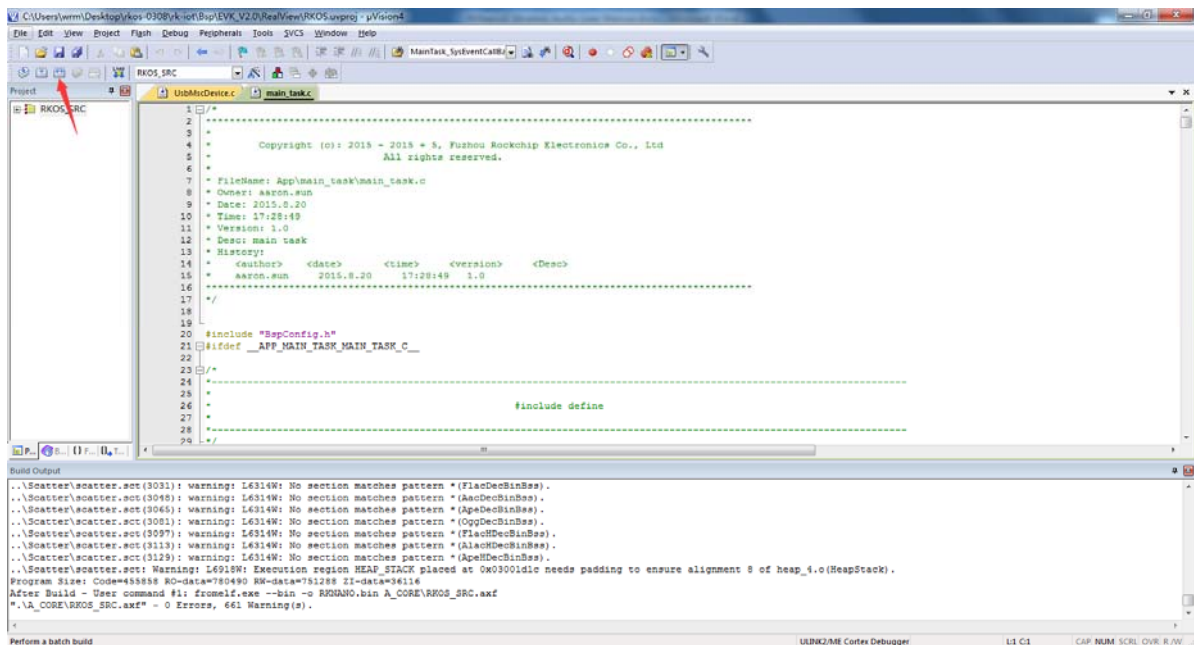
```
_BSP_EVK_V20_BSP_COMMON_
COMMON API void GetAirplayDeviceName(uint8 * name)
{
    strcpy(name, "RKNanoD_Device");
}
```

Bluetooth: ..\Bsp\EVK_V2.0\Bsp.c 中的 GetBtAudioDeviceName ()函数中, 修改以下部分:

```
_BSP_EVK_V20_BSP_COMMON_
COMMON API void GetBtAudioDeviceName(uint8 * name)
{
    #ifdef ENABLE_NFC
        strcpy(name, "nft_test");
    #else
        strcpy(name, "RKNanoD_BT");
    #endif
}
```

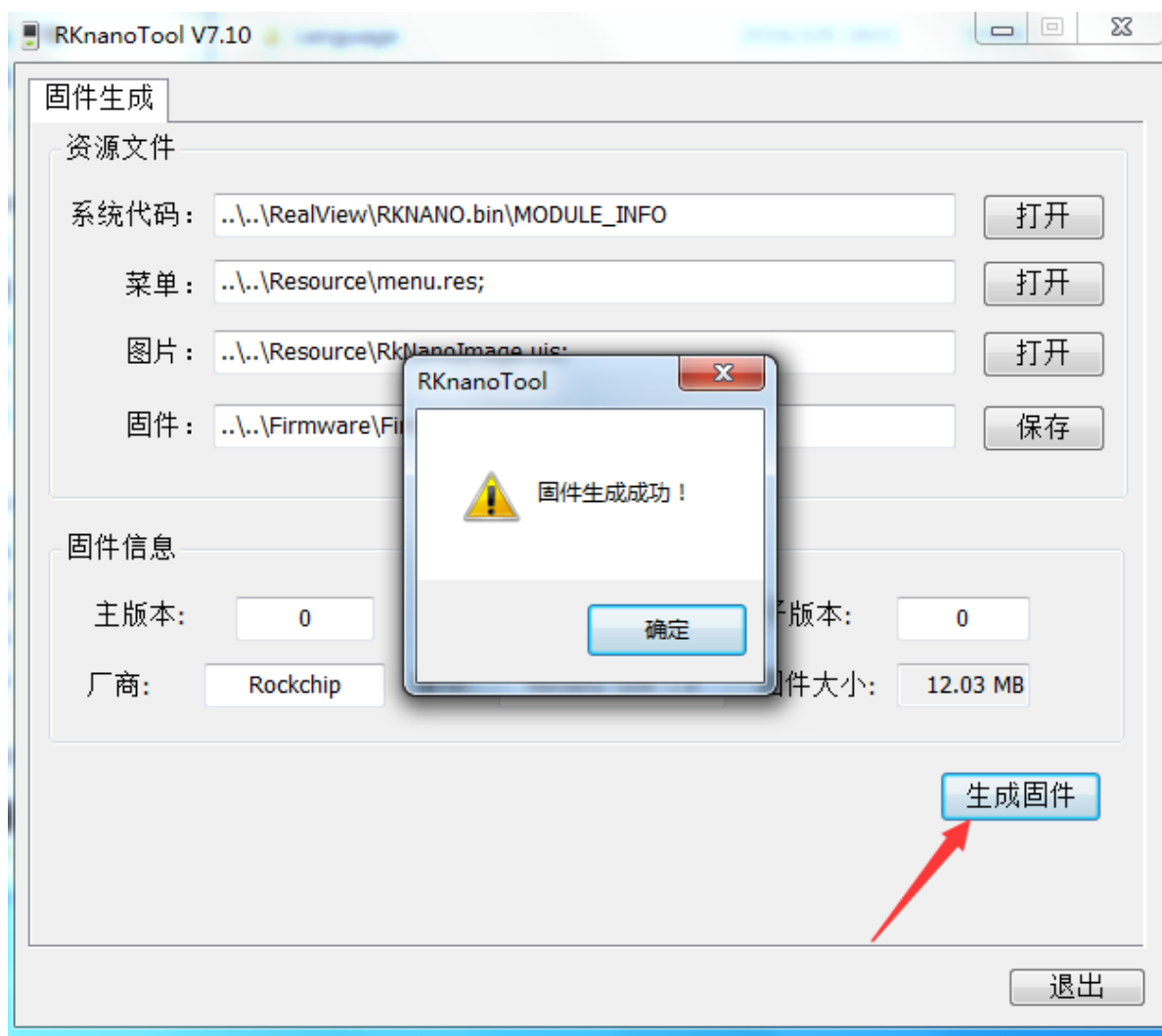
4.2 SDK 编译

RKNanoD Wireless Audio SDK 支持 RealViewMDK 编译, 打开工程目录下的 Bsp\EVK_V2.0\RealView\ RKNanoD.uvproj, 编译工程, 如图:



4.2 固件生成

RKNanoD Wireless Audio SDK 的固件打包生成, 需要使用 Rockchip 开发的固件打包生成工具, 如下图所示:



由于 SDK 支持 eMMC 和 SPI Flash, 以及由于 SPI Flash 本身容量大小原因, 为了方便客户开发调试, RK 将 eMMC 和 SPI 的固件打包工具分别放在不同的目录中:

EMMC Flash: Bsp\EVK_V2.0\Development\firmware_generate_eMMC\RKNanoTool.exe

SPI Flash: Bsp\EVK_V2.0\Development\firmware_generate_SPI\RKNanoTool.exe

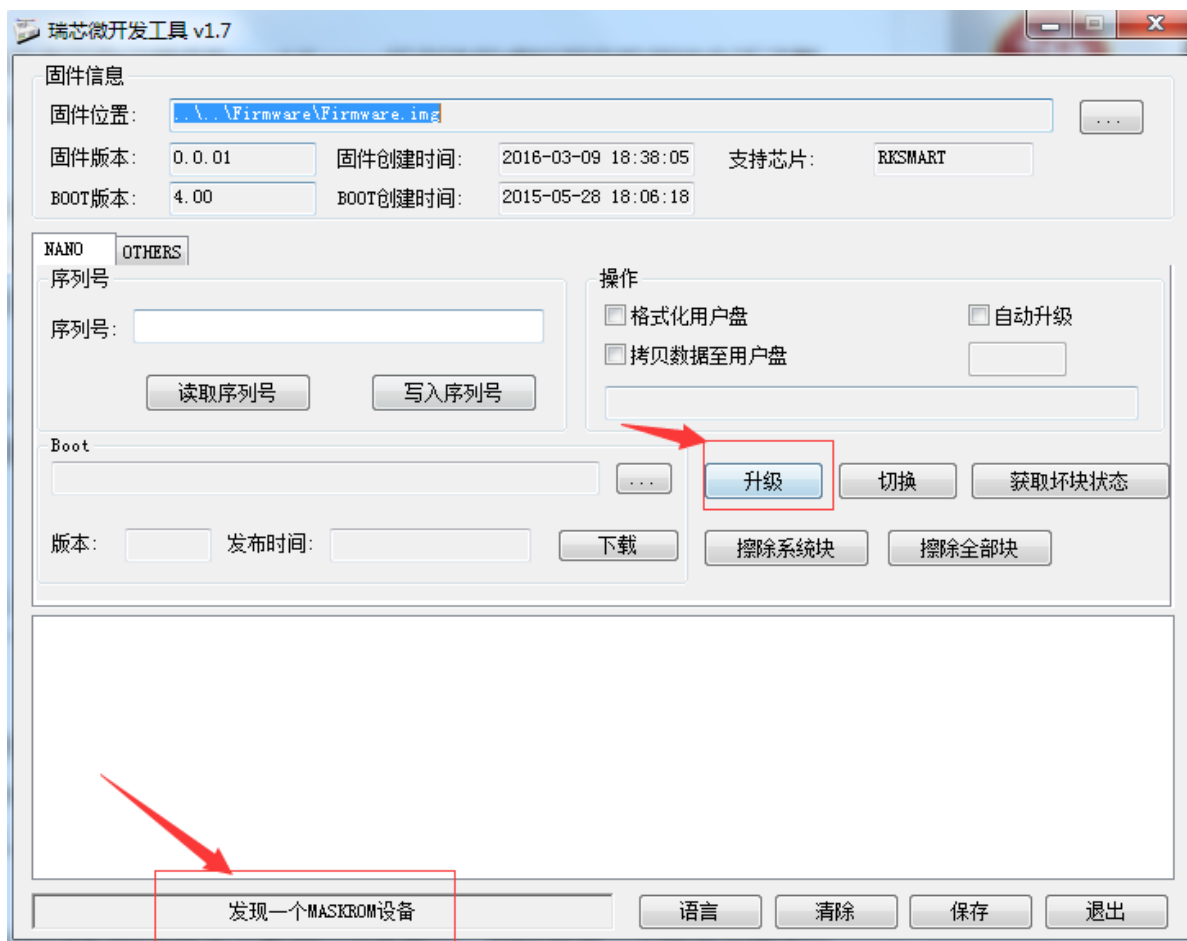
区别在于 SPI 固件打包时, 由于其容量大小限制, 在固件打包时将一些资源数据未打包在固件中, 调试时需要将其拷贝至 SD 卡中运行。而 eMMC 本身容量比较充足, 资源文件和软件代码同时打包在固件中。

4.3 固件下载

RKNanoD Wireless Audio SDK 的固件下载需要使用 Rockchip 开发的固件下载工具进行升级，目前仅支持 USB 升级模式，固件下载工具为：

Bsp\EVK_V2.0\Development\firmware_upgrade\RKDevelopTool.exe

操作方式如下图：



注意：若用 SPI Flash 进行 UI 显示调试时，需要将以下未打包到固件的资源拷贝到 SD 卡下的根目录中。原则上 SPI Flash 的配置不支持 UI 功能。

这些文件可以在如下路径中找到：

Bsp\EVK_V2.0\Development\firmware_generate_eMMC\Font12.bin

Bsp\EVK_V2.0\Development\firmware_generate_eMMC\Font16.bin

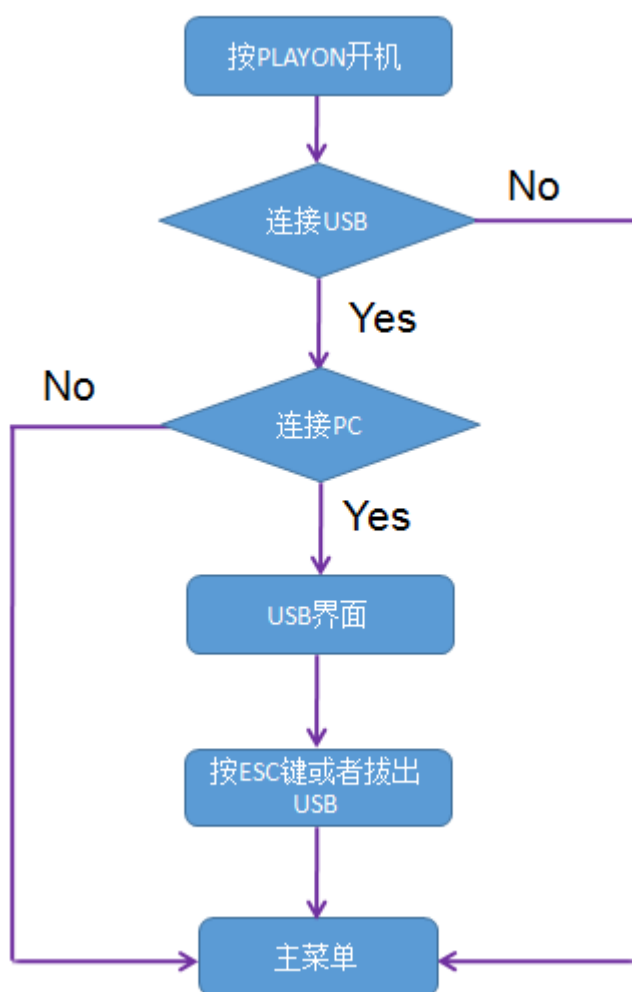
Bsp\EVK_V2.0\Development\firmware_generate_eMMC\CodePage.bin

Bsp\EVK_V2.0\Resource\menu.res

Bsp\EVK_V2.0\Resource\RkNanoImage.uis

5 SDK 操作指南

5.1 开机流程



5.2 操作说明

5.2.1 连接 WIFI

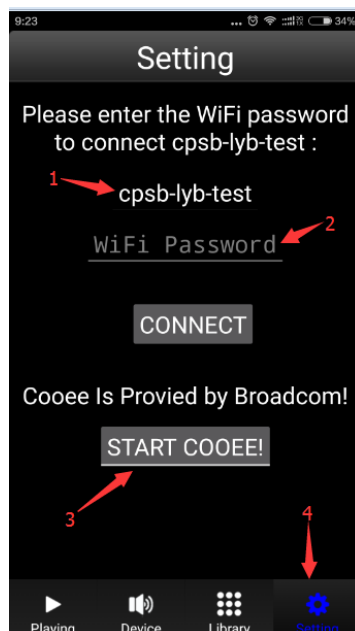
如果是初次连接或者更改路由器，首先需要进行 WiFi 连接路由器操作；如果上次 WiFi 已经连接成功，打开 WiFi 后设备后自动连接上次连接过的路由器。操作步骤如下：

初次连接步骤如下：

进入设置->WLAN->WiFi 开关->开启->等待 WiFi 打开->等待对话框消失

进入设置-> WLAN->路由器配置->等待手机推送 WiFi 路由器名称及密码

在设备进入路由器配置模式之后，手机端需要打开 WiFi 路由器推送的 APP 软件，进行路由器和密码配置，推送成功后，设备上的对话框将会自动消失。APP 软件界面如下图所示：



箭头 1：表示当前手机连接的 WiFi；

箭头 2：输入当前连接 WiFi 的密码；

箭头 3：当输入完密码以后，按 START COOEE 进行推送；

重新连接步骤如下：

进入设置->WLAN->WiFi 开关->开启->等待 WiFi 打开->等待对话框消失

WiFi 打开后，会自动重新连接上次记忆的路由器，同时会显示目前设备搜索到的所有路由器设备，

其中第一个路由器名称为当前连接路由器。

注意：蓝牙与 WiFi、本地音乐播放器不能同时打开，因此在进行打开 WiFi 时要先关闭蓝牙和音乐播放器；

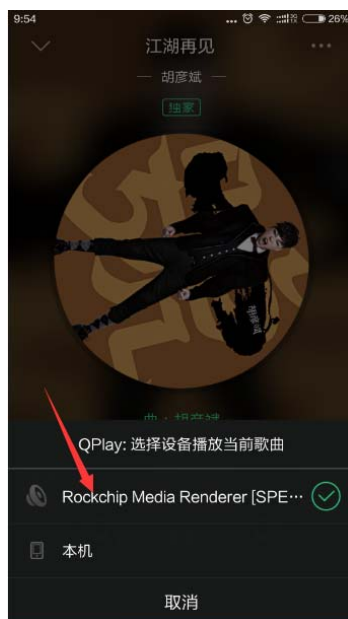
5.2.2 DLNA 播放器操作

WiFi 连接成功之后，设备顶端的状态栏会显示 WiFi 已连接图标。通过按键返回到主界面，选在“DLNA 播放”进入 DLNA 播放界面，启动 DLNA 播放器功能，此时手机播放器上会显示到发

现 DLNA 设备图标，如下图 QQ 音乐所示：



点击如上图箭头所指图标，选择 DLNA 设备，开始将手机音乐推送到设备进行播放。



长按“ESC”键退出 DLNA 播放器功能。

注意：目前 DLNA 播放器功能仅支持 MP3 音乐文件推送；

5.2.3 第三方播放器操作

SDK 提供一些第三方播放器的接口，用户可以根据自己的需要，在满足要求的情况下添加自己的播放器，例如添加 AIRPLAY 播放器，Airplay 播放器功能需要苹果设备才能使用，在苹果设备上会显示到发现 Airplay 设备图标，如下图 QQ 音乐所示：



点击如上图箭头所指图标，选择 Airplay 设备，开始将手机音乐推送到设备进行播放。

5.2.4 蓝牙播放器操作

RKNanoD Wireless Audio SDK 同时可以支持蓝牙播放器功能，蓝牙播放器的操作方法如下：

进入设置->蓝牙->开启->等待蓝牙打开->对话框消失；

进入主界面->蓝牙播放

SDK 目前支持 A2DP、AVRCP 功能。蓝牙打开之后，可以通过手机端搜索蓝牙设备，连接之后即可进行蓝牙音乐播放功能。

蓝牙播放的关闭需要从设置菜单中关闭蓝牙。

注意：蓝牙与 WiFi、本地音乐播放器不能同时打开

5.2.5 本地播放器操作

RKNanoD Wireless Audio SDK 同时可以支持本地磁盘音乐文件播放功能，在主菜单界面选择“音乐播放”或者“资源管理器”进入文件浏览界面，选择文件后进入本地播放功能。

长按“ESC”键结束本地播放功能。

5.3 无屏音箱配置与操作

为了满足客户对无线音箱不带屏幕的需求，我们提供一份无屏音箱的解决方案，此方案取消了 GUI 和屏幕的代码，增加了代码的空间，从而使得客户的应用程序有了更多的发挥空间。

5.3.1 无屏音箱代码配置

WIFI 无屏音箱配置：

```
/*
 *
 * Function config
 */
*****
// #define _USE_GUI_ /* Enable GUI */
#define _USE_GUI_ /* ----GUI config----- */
/* Enable Backlight */
#define _BACKLIGHT_
/* Enable LED in no screen mode */
#define NOSCREEN_USE_LED
#endif

#define _WIFI_ /* Enable WIFI */
#define _WIFI_ /* ----WIFI config----- */
#define _WIFI_OB /* Enable WIFI IO interrupt */
#define _WIFI_FOR_SYSTEM /* Enable save WIFI information */
// #define _WIFI_5G_ap6234 /* Enable WIFI 5G Module */
#endif

// #define _BLUETOOTH_ /* Enable Bluetooth */
```

蓝牙无屏音箱配置：

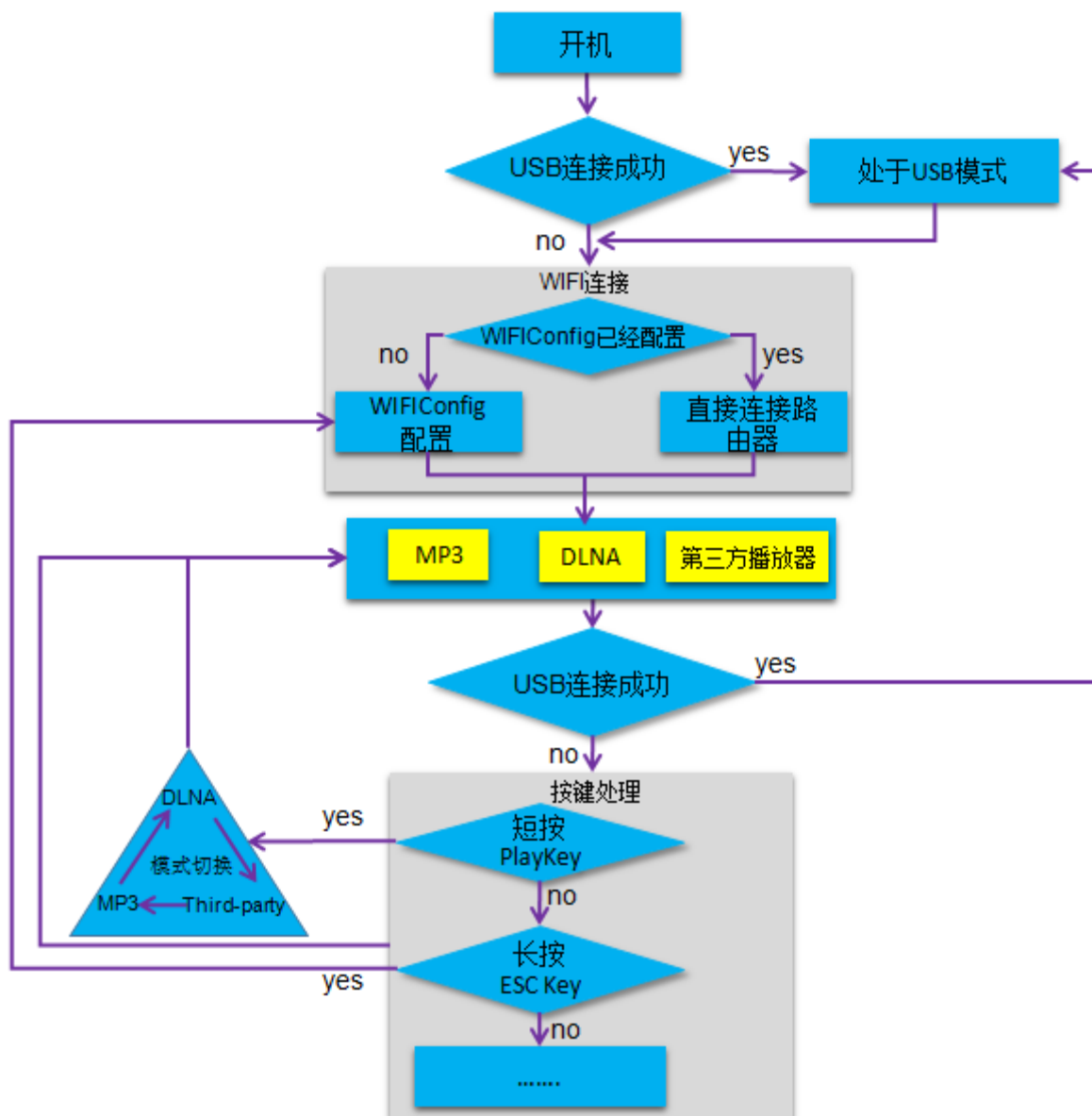
```
/*
 *
 * Function config
 */
*****
// #define _USE_GUI_ /* Enable GUI */
#define _USE_GUI_ /* ----GUI config----- */
/* Enable Backlight */
#define _BACKLIGHT_
/* Enable LED in no screen mode */
#define NOSCREEN_USE_LED
#endif

// #define _WIFI_ /* Enable WIFI */
#define _WIFI_ /* ----WIFI config----- */
#define _WIFI_OB /* Enable WIFI IO interrupt */
#define _WIFI_FOR_SYSTEM /* Enable save WIFI information */
// #define _WIFI_5G_ap6234 /* Enable WIFI 5G Module */
#endif

#define _BLUETOOTH_ /* Enable Bluetooth */
```

1. 如上图所示，修改 Bsp\EVK_V2.0\BspConfig.h 中的宏定义；
2. 配置完成以后重新编译代码，下载固件即可；
3. 注意无屏音箱中 WIFI 和蓝牙目前是不共存的，不能同时定义；

5.3.2 WIFI 无屏音箱操作流程



由上图可知：

- 1.WIFI 无屏音箱支持三种播放器：MP3 播放器，DLNA 播放器，第三方播放器接口；
- 2.USB 模式：无屏音箱只能作为一个存储器；
- 3.按键操作如下表所示：

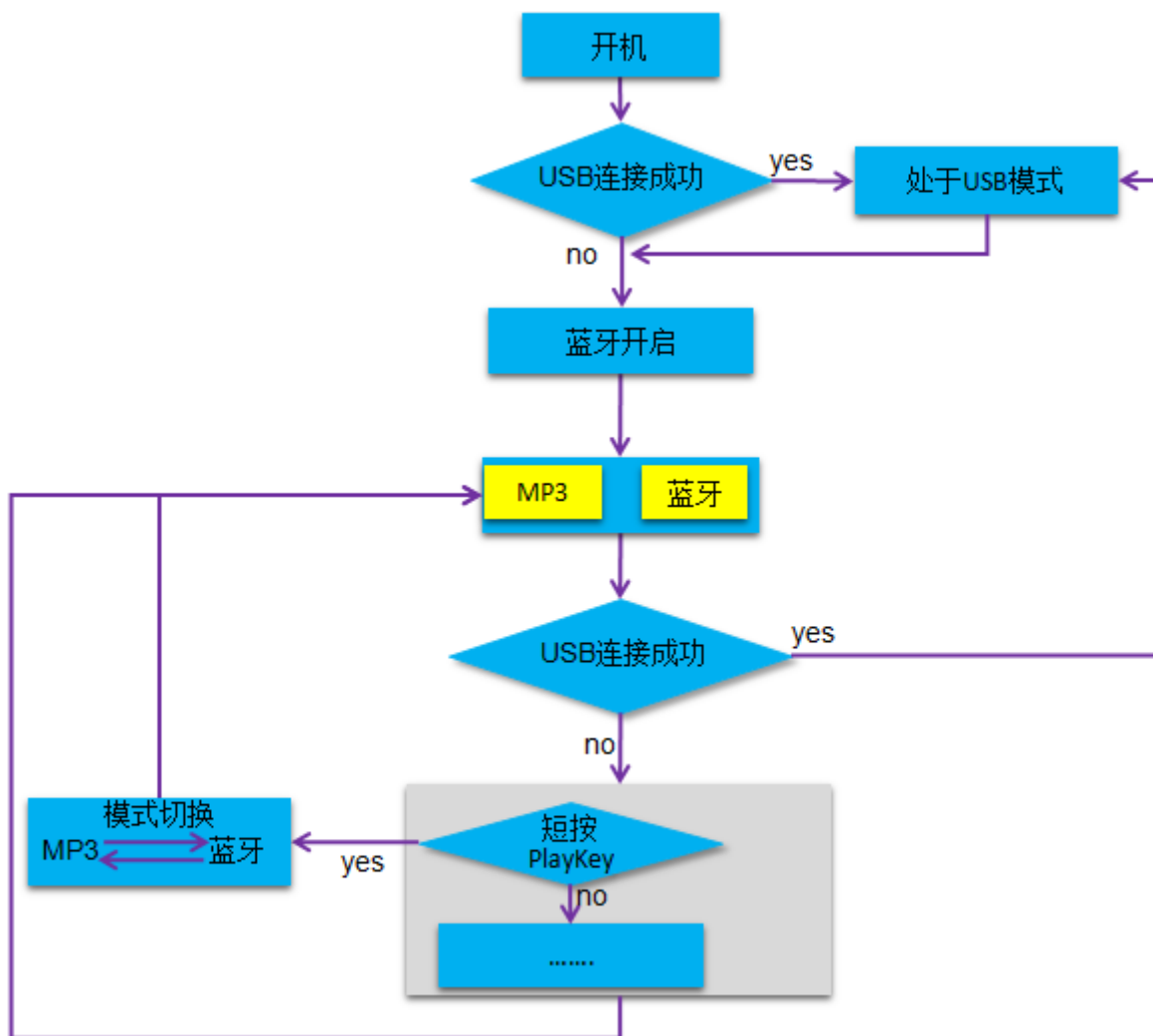
按键一般操作（MP3 播放器）	
Menu 键	播放和暂停
UP 键	音量加
DOWN 键	音量减

FFW 键	短按下一曲，长按快进
FFD 键	短按上一曲，长按快退
PLAYON 键	短按切换播放器
ESC 键	长按开启或者停止 WIFIconfig 配置

4.若使用 LED 灯作为状态标识，如下表：

LED 状态标志		
系统状态	LED（红）	LED(蓝)
USB 模式	--	--
MP3	ON	OFF
DLNA	ON	ON
DLNA 连接中	ON	FLASH
第三方播放器	OFF	ON
第三方播放器连接中	OFF	FLASH
WIFIconfig 配置中	FLASH	FLASH

5.3.3 蓝牙无屏音箱操作流程



由上图可知：

1. 蓝牙无屏音箱支持两种播放器：MP3 播放器，蓝牙播放器；
2. USB 模式：无屏音箱只能作为一个存储器；
3. 按键操作如下表所示：

按键一般操作（MP3 播放器，蓝牙播放器）	
Menu 键	播放和暂停
UP 键	音量加
DOWN 键	音量减
FFW 键	短按下一曲，长按快进（快进蓝牙不支持）
FFD 键	短按上一曲，长按快退（快退蓝牙不支持）

PLAYON 键	短按切换播放器
----------	---------

4.若使用 LED 灯作为状态标识，如下表：

LED 状态标志		
系统状态	LED（红）	LED(蓝)
USB 模式	--	--
MP3	ON	OFF
蓝牙播放器	OFF	ON
蓝牙连接过程中	OFF	FLASH

6 SDK 辅助工具

6.1 shell 命令集合

6.1.1 音乐 shell 命令

Shell 命令	命令说明
connect music	连接 music 的 shell 命令，连接以后能对 music 进行 shell 操作
music.create	创建 music
music.delete	删除 music
remove music	移除 music 的 shell 连接
music.help	music 命令帮助指南
music.play	音乐播放
music.stop	音乐暂停
music.up	音乐上一曲
music.next	音乐下一曲
music.ffw	快进
music.ffd	快退
music.ep	EQ 模式
music.vol	音量设置
music.rp	循环模式

6.1.2 USB shell 命令

Shell 命令	命令说明
connect usbotg	连接 usbotg 的 shell 命令，连接以后能对 usbotg 进行 shell 操作
usbotg.create	创建 usbotg
usbotg.delete	usbotg 删除
remove usbotg	移除 usbotg 的 shell 连接

connect usbmsc	连接 usbmsc 的 shell 命令，连接以后能对 usbmsc 进行 shell 操作
usbmsc.create	创建 usbmsc
usbmsc.test	Usbmsc 连接测试
usbmsc.delete	usbmsc 删除
remove usbmsc	移除 usbmsc 的 shell 连接

6.1.3 WIFI、DLNA、shell 命令

Shell 命令	命令说明
connect wifi	连接 wifi 的 shell 命令，连接以后能对 wifi 进行 shell 操作
wifi.create	创建 wifi
wifi.delete	wifi 删除
remove wifi	移除 wifi 的 shell 连接
connect dlna	连接 dlna 的 shell 命令，连接以后能对 dlna 进行 shell 操作
dlna.start	开始 dlna
remove dlna	移除 dlna 的 shell 连接
dlna.delete	删除 dlna

6.1.4 蓝牙 shell 命令

Shell 命令	命令说明
connect bt	连接 bt 的 shell 命令，连接以后能对 bt 进行 shell 操作
bt.open	创建 bt
bt.close	删除 bt
remove bt	移除 bt 的 shell 连接

6.1.4 系统 shell 命令

Shell 命令	命令说明
----------	------

connect system	连接 system 的 shell 命令，连接以后能对 system 进行 shell 操作
system.memory	查看系统内存
system.rkos	系统运行状态
system.freq	查看系统频率测试（暂时不用测试）
system.cpu	系统 cpu 测试（暂时不用）
remove system	移除 system 的 shell 连接

6.1.5 线程 shell 命令

Shell 命令	命令说明
connect task	连接 task 的 shell 命令，连接以后能对 task 进行 shell 操作
task.list	查看当前存在线程
remove task	移除 task 的 shell 连接

6.2 shell 命令调试实例

6.2.1 音乐播放 shell 操作

注意：不能在启动 USB 存储的情况下创建音乐，因为 USB 占用了大量内存,所以应该在退出 USB 存储的情况下调试

在正常开机以后，若开发板连接了串口线，可以通过串口发命令进行调试，如图：

```
[A]audio create ok
[A][000002.27]SegmentID = 52
[A][000002.27]SegmentID = 51
[A][000002.28]create thread classId = 32, objectid = 0,remain = 240168
[A][000002.28]SegmentID = 58
[A][000002.28]SegmentID = 57
[A][000002.29]list malloc success

[A][000002.29]create thread classId = 42, objectid = 0,remain = 216960welcome to use RKOS
RKOS V1.0.0 - Copyright (C) 2018 Fuzhou Rockchips Electronics CO.,Ltd
Please visit http://www.rock-chips.com to ensure you are using the last version
*****
*
*   RKOS SHELL V1.0.0
*   Copyright (C) 2018 Fuzhou Rockchips Electronics CO.,Ltd
*
*   >>> See the program guide of project for details. <<<
*   If you do not how to use RKOS, please input help cmd.
*   also send message to aaron.sun@rock-chips.com
*   Thank you for using RKOS, and thank you for your support!
*
*****
cur system is AP system

[000002.39]vco=0, pll=24000000, shclk=24000000, stck=24000000, splk=24000000, hclk=1000000, htck=1000000
[000002.45]
rkos://
rkos://■
```

若开机后没有通过 UI 界面打开音乐播放器，可以通过 shell 命令行的方式启动音乐，如图：

```
rkos://
rkos://connect music
[A][000122.56]SegmentID = 25
device connect success
rkos://music.create
[A][000126.63]SegmentID = 68
[A][000126.64]SOURCE_FROM_FILE_BROWSER

[A][000126.64]total music = 4
[A][000126.64]FREQ_ExitModule

[A][000126.65]RegMBoxDecodeSvc

[A][000126.66]create thread classId = 34, objectId = 0,remain = 153544
[A][000126.67]audio control task enter...
rkos://
[A][000126.68]SegmentID = 66
[A][000126.68]create thread classId = 33, objectId = 0,remain = 145016
[A][000126.69]FILE: ..\..\..\App\Audio\AudioControlTask.c, LINE: 383: Audio Start
[000126.69]vco=432000000, pll=144000000, shclk=72000000, stck=72000000, splk=72000000, l
[000126.70]
[A][000126.70]FILE: ..\..\..\App\Audio\AudioControlTask.c, LINE: 3823: CurrentDecCodec :
[A][000126.69]stream control enter...
[A][000126.71]audio auto anlyse == maybe no.id3.mp3, framesize = 0
[A][000126.72]audio auto anlyse == no.id3.mp3
[A][000126.73]short name:姚思思~1MP3
[A][000126.73]SegmentID = 74
[A][000126.73]SegmentID = 205
[A][000126.76]FILE: ..\..\..\Driver\Bcore\BcoreDevice.c, LINE: 207: start bb system...
[B][000000.00]chip_freq2.hclk_cal_core =48000000!
```

若要结束播放的歌曲，可以按下图方式操作：

```
[A][000403.74]channel=2 bitpersample = 16 fs = 44100 bitrate = 128000
[A][000403.75]FILE: ..\..\..\App\Audio\AudioControlTask.c, LINE: 431

[000403.76]vco=0, pll=240000000, shclk=240000000, stck=240000000, splk=;
[000403.76]
[000403.76]vco=480000000, pll=480000000, shclk=480000000, stck=480000000
[000403.77]
[A][000403.77]16 bit len =4608channel LR=2
[000403.77]
[A][000403.77]>>>bitpersample =16CodecFS =44100
[000403.79]
[A][000403.87]AudioStop over ReqType = 0, cursate = 3

rkos://music.delete
[A][000430.46]delete thread classId = 34, objectId = 0, remain = 140;
[A][000430.47]AudioControlTask_DeInit

[A][000430.47]Audio Stop,ReqType
[B][000000.00]AUDIO_DECODE_CLOSE
[A][000430.61]FILE: ..\..\..\Driver\Bcore\BcoreDevice.c, LINE: 140: ;
[000430.62]vco=0, pll=240000000, shclk=240000000, stck=240000000, splk=;
[000430.62]
[A][000430.62]AudioStop over ReqType = 2, cursate = 3
[A][000430.73]delete thread classId = 33, objectId = 0, remain = 216;
[000430.73]audio delete ok
rkos://remove music
device remove success
rkos://
```


这样就完成了不进行 UI 操作的情况下，执行音乐播放功能。shell 命令是用来在没有 UI 界面的情况下调试代码的，善于利用 shell 调试可以加快开发进度。